SAÉ 3.2 — Vérification et maintenance de la partie puissance et commande d'un système de conversion de l'énergie

Projet : robot auto-équilibré ELEGOO Tumbller — mise en service, redressement autonome et coopération maître/suiveur

Département Génie Électrique et Informatique Industrielle

BUT GEII — Semestre 3

24 h de projet (8 séances de 3 h) — Travail en groupes



Plan de la présentation

Cadre pédagogique : une situation professionnelle

Le système support : robot ELEGOO Tumblller

La mission : trois jalons

Organisation, livrables et évaluation

Partie 1

Cadre pédagogique : une situation professionnelle

- Fiche nationale : **SAÉ 3.EME.02** — « Vérification et maintenance de la partie puissance et commande d'un système de conversion de l'énergie » (programme national BUT GEII, arrêté du 15 avril 2022).
- **Situation professionnelle authentique** : un système existant vous est remis « sur étagère », avec son code d'origine, mais sans documentation de vérification ni de maintenance.
- **Mission double** :
  - **remettre en service de manière documentée** : vérification de la partie puissance et de la partie commande, épuration du code ;
  - **faire évoluer le système** : redressement autonome, puis mise en réseau de deux robots coopérant en mode maître/suiveur.

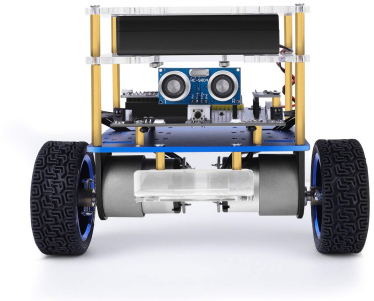
Compétence	Niveau	Mise en œuvre
C2 — Vérifier la partie GEII d'un système	2	Protocoles de tests et mesures sur la chaîne de puissance et la chaîne d'information ; interprétation ; analyse des dysfonctionnements.
C3 — Maintenir en condition opérationnelle	1	Procédure de maintenance ; ajout d'entrées/sorties (ESP32) et mise à jour du programme.
C1 — Concevoir la partie GEII (mobilisée)	2	Redressement autonome, protocole maître/suiveur, machine à états.
C4 — Installer, convertir et gérer l'énergie (mobilisée)	1	Intégration matérielle ESP32, alimentation, câblage, liaison radio.

Remarque

Les compétences transversales (gestion de projet, Git, communication écrite et orale) sont évaluées tout au long du projet.

Partie 2

Le système support : robot ELEGOO Tumblller



Robot ELEGOO Tumblller assemblé (photo produit ELEGOO).

Le Tumblller est un robot **auto-équilibré sur deux roues** : il résout en permanence le problème du *pendule inverse* en mesurant son inclinaison et en agissant sur la vitesse des moteurs, à cadence fixe (200 Hz).

C'est une **chaîne de conversion d'énergie complète** :

- accumulateur Li-ion 18650 (3,7 V) ;
- convertisseur 5 V ;
- double pont en H ;
- deux moteurs CC à réducteur, pilotés en PWM.

Fonction	Élément	À relever ?
Commande	Arduino Nano (ATmega328P, 5 V) sur carte d'extension ELEGOO	—
Mesure d'attitude	GY-521 : centrale inertielle MPU6050, bus I^2C	—
Détection d'obstacles	Ultrasons (face avant) + 2 infrarouges latéraux	portée utile
Puissance	Li-ion 18650, convertisseur 5 V, double pont en H	courant, rendement
Communication d'origine	Bluetooth « BT16 » (application mobile), infrarouge	—
Interfaces	Bouton, LEDs RVB, interrupteur	—

Résultats à fournir (dès les séances 1–2)

Schéma **fonctionnel** (chaînes d'information et d'énergie) et schéma **structurel** du système : livrables de référence pour tout le projet.

- **Interruption cadencée à 5 ms (200 Hz)** :
  - lecture de la centrale inertielle, filtrage (filtre de Kalman) ;
  - régulations : PD d'angle, PI de vitesse, PD de rotation ;
  - génération des commandes PWM des moteurs.
- **Boucle principale** : bouton, Bluetooth, ultrasons, infrarouges, tension batterie, LEDs, comportements autonomes (suivi, évitement, rebond).

Remarque

Le code constructeur est fonctionnel mais *monolithique* : un seul fichier, des fonctions qui ne serviront pas votre mission. Son **épuration** est l'objet du jalon 1 — sans jamais dégrader l'équilibre.

Avertissement sécurité

- **Accumulateur Li-ion** : jamais en court-circuit, jamais percé ni démonté ; charge sur le chargeur fourni, sous surveillance ; débranché avant tout câblage.
- **Toujours couper l'alimentation** avant de modifier le câblage (y compris pour l'intégration des ESP32).
- **Les roues tournent** dès la mise en équilibre : robot hors sol aux premiers essais ; dégager la zone d'essais.
- **ESP32 en logique 3,3 V**, Nano en 5 V : pont diviseur ou adaptateur de niveau **obligatoire** sur la voie Nano → ESP32.
- Précautions usuelles : alimentations limitées en courant, multimètres vérifiés, décharges électrostatiques.

Tout manquement grave à la sécurité entraîne la non-validation de la séance.

Partie 3

La mission : trois jalons

Jalon 1 (séances 1 à 4) — Remise en service documentée

Réception et essais d'origine ; vérification de la partie puissance et commande ; récupération du projet, audit du code ; épuration et refonte modulaire avec tests de non-régression.

Jalon 2 (séance 5) — Redressement autonome

Le robot, couché sur le dos, doit **se mettre debout tout seul** après un simple ordre, puis repasser en mode équilibre, sans intervention mécanique.

Jalon 3 (séances 6 et 7) — Coopération maître/suiveur

Deux robots communiquent par radio (ESP32 ajoutés, protocole ESP-NOW) : le **maître** détecte les obstacles et **ordonne** au **suiveur** d'avancer, s'arrêter, tourner ou se relever ; arrêt sécurisé en cas de perte de liaison.

Séance 1 — réception :

- inventaire, contrôle visuel, procès-verbal de réception ;
- relevé des références des composants (microcontrôleur, pont en H, convertisseur) ;
- essais de *toutes* les fonctions usine : tableau conforme / non conforme.

Séance 2 — mesures sur la chaîne de puissance :

- tension batterie au repos / sous charge / en poussée maximale ;
- courant et puissance appelés dans chaque état ;
- signaux PWM et tension moteur à l'oscilloscope ;
- ondulation du bus 5 V ; autonomie énergétique extrapolée.

Résultats à fournir

PV de réception, schémas fonctionnel et structurel, tableau de mesures (V_{bat}, I, P), PV d'essais des fonctions d'origine.

Séance 3 — comprendre avant de modifier :

- compiler et rejouer le comportement d'usine du firmware officiel ;
- **cartographier** : rôle de chaque fonction, localisation de l'ISR 5 ms ;
- **auditer** : fonctions inutiles, valeurs non nommées, duplications ;
- rédiger le **plan d'épuration** et les tests de non-régression.

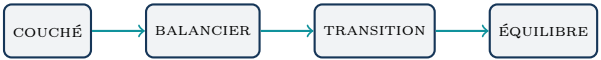
Séance 4 — épurer, sans casser :

- réorganisation en modules (IMU, équilibre, moteurs, capteurs, communication) ;
- suppression par étapes, **tests rejoués à chaque étape** ;
- versionnage Git : toute régression doit pouvoir être annulée.

Remarque

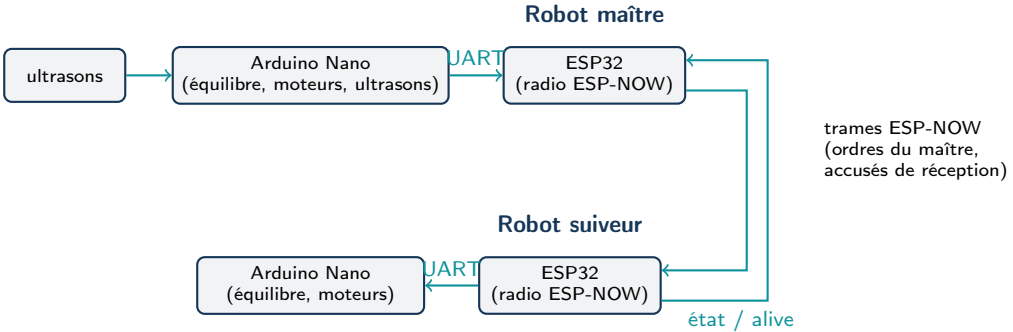
Épurer ne veut pas dire « faire plus court », mais **ne conserver que ce qui sert la mission**, de façon lisible, testable et vérifiable.

- Un robot à deux roues couché est un pendule que l'on peut faire basculer en accumulant de l'énergie cinétique par un mouvement de **balancier**.
- Machine à états à spécifier :



- Critères de passage **mesurés sur l'angle** ; repli en cas d'échec (nombre de tentatives borné).
- La manœuvre s'insère dans la boucle d'asservissement : **ne jamais bloquer l'interruption 5 ms**.
- Réglages expérimentaux ; relevé d'angle d'un relevage réussi et d'un échec.

Jalon 3 — Architecture maître/suiveur (séances 6–7)



- Le maître **perçoit** (ultrasons) et **décide** ; le suiveur **exécute** en conservant son équilibre.
- Câblage : ESP32 alimenté par le bus 5 V (broche `VIN`), masses communes, **adaptation de niveau 5 V → 3,3 V** obligatoire.

Trame compacte échangée (à adapter) :

```
// Trame echangee Nano <-> ESP32 puis ESP-NOW (4 octets)
typedef struct __attribute__((packed)) {
  uint8_t sync;   // octet de synchronisation (ex. 0xA5)
  uint8_t cmd;    // ALIVE / STANDUP / MOVE_F / MOVE_B / STOP / TURN_L / TURN_R
  uint8_t speed;  // consigne de vitesse (0..255)
  uint8_t crc8;   // somme de controle des octets precedents
} trame_t;
```

Failsafe — le suiveur ne roule que si le maître est actif :

```
if (millis() - dernier_ordre_valide > 500) {
  mode = ARRET_SECURISE;   // perte de liaison -> stop
}
```

Remarque

L'UART est gérée dans la boucle principale à cadence faible (10 Hz), jamais dans l'interruption 5 ms. Toute trame invalide (CRC) est ignorée.

Exigences chiffrées (critères d'acceptation)

Réf.	Exigence	Valeur
EX1	Équilibre statique conservé après épuration du code	$\geq$ 60 s sans chute
EX2	Redressement autonome depuis la position couchée	$\leq$ 15 s, $\leq$ 3 essais
EX3	Portée de la liaison maître/suiveur en salle de TP	$\geq$ 10 m
EX4	Latence moyenne décision du maître $\rightarrow$ action du suiveur	$\leq$ 200 ms
EX5	Perte de liaison $\Rightarrow$ arrêt sécurisé du suiveur	$\leq$ 500 ms
EX6	Toute trame invalide (CRC) est ignorée	0 action sur trame corrompue
EX7	Toute commande radio tracée côté récepteur	100 % des trames

Scénario de la démonstration finale (séance 8)

1. Les deux robots sont **couchés** sur le sol ; ordre de démarrage au maître.
2. Le maître **se redresse seul**, puis ordonne au suiveur de se redresser (EX2).
3. Le suiveur démarre ; les deux robots roulent **en convoi**.
4. Un obstacle est placé devant le maître : détection ultrasons, arrêt du maître et ordre d'arrêt au suiveur (EX4, EX6).
5. L'obstacle est retiré : **reprise de la marche** des deux robots.
6. L'enseignant coupe la radio du maître : le suiveur passe en **arrêt sécurisé** (EX5) et se stabilise sans tomber.

Partie 4

Organisation, livrables et évaluation

Planning des 8 séances (24 h encadrées)

Séance	Contenu	Jalon / revue
S1	Réception, inventaire, essais du firmware d'origine	J1
S2	Mesures puissance/commande, schémas fonctionnel et structurel	J1
S3	Récupération du projet, compilation, audit, cartographie	J1
S4	Épuration, refonte modulaire, non-régression	**Revue J1**
S5	Redressement autonome	**Revue J2 : démo EX2**
S6	Intégration ESP32, protocole, essais ESP-NOW	J3
S7	Mission maître/suiveur, failsafe, scénario complet	**Revue J3**
S8	Vérification finale, limites, maintenance, soutenance	**Démo finale**

Chaque revue de jalon : point d'avancement de 10 minutes avec l'enseignant, décisions consignées dans le dépôt Git ; l'autonomie d'organisation fait partie de l'évaluation.

Livrables (récapitulatif)

1. **Schéma fonctionnel et schéma structurel** du système.
2. **Dossier de vérification** : protocoles de test, résultats de mesures, tests de non-régression, campagne finale.
3. **Dossier de maintenance** : procédures de remplacement, mise à jour du firmware, arbre de diagnostic.
4. **Analyse des limites de fonctionnement** (dérisquage).
5. **Code complet** (firmware épuré des deux rôles, code ESP32), versionné dans le dépôt Git du groupe.
6. **Rapport final** (de préférence en LaTeX : modèles du département fournis) et soutenance avec démonstration.

Évaluation : grille sur 20 points

Réf.	Élément évalué	Points
V1	Réception, PV, essais d'origine, schémas fonctionnel et structurel	2
V2	Vérification partie puissance et commande (mesures, protocoles, autonomie)	2
V3	Audit et épuration du firmware (carte du code, refonte, EX1)	4
V4	Redressement autonome (EX2) : spécification, réglages, démonstration	3
V5	Liaison ESP-NOW : protocole, portée et latence (EX3, EX4, EX6)	3
V6	Mission maître/suiveur et failsafe (EX5, EX7), scénario complet	3
V7	Dossiers de vérification, de maintenance et analyse des limites	2
V8	Rapport final et soutenance	1
Total		**20**

Critères complémentaires : unités et incertitudes ; tout écart à une exigence doit être *analysé* (cause, criticité, action corrective) et non dissimulé.

Remarque — vos premières actions (séance 1)

- constituer le groupe, prendre possession du robot et du dépôt Git ;
- inventaire complet du colis, contrôle visuel, PV de réception ;
- mise sous tension et essais des fonctions usine du firmware d'origine.

Bon projet à tous !



Département Génie Électrique et Informatique Industrielle — BUT GEII